

Teoría de Arreglos:

Un arreglo es una estructura de datos que almacena una colección de elementos del mismo tipo bajo un solo nombre. Cada elemento en el arreglo tiene un índice único que indica su posición. El índice suele comenzar en cero para el primer elemento y se incrementa en uno para cada elemento subsiguiente.

Diferencias con Otros Identificadores:

- Variables: Almacenan un único valor, mientras que los arreglos almacenan múltiples valores del mismo tipo.
- Constantes: Al igual que las variables, almacenan un único valor constante.
- Contadores y Acumuladores: Son variables simples utilizadas para contar o acumular valores, no almacenan una colección de datos.

Importancia de los Arreglos:

- Organización de Datos: Permiten organizar datos de manera estructurada.
- Acceso Eficiente: Facilitan el acceso y manipulación de datos mediante índices.
- Iteración Simplificada: Se utilizan en bucles para procesar elementos de manera eficiente.

Operaciones Básicas:

Declaración e Inicialización:

```
definir arregloEnteros[5] como entero; // Declaración de un arreglo de enteros con 5 elementos
arregloEnteros = [1, 2, 3, 4, 5]; // Inicialización del arreglo
```

Recorrido:

```
para i desde 0 hasta longitud(arregloEnteros) - 1 con paso 1
    escribir arregloEnteros[i];
fin para
```

Insertión:

```
// Insertar un nuevo elemento al final del arreglo
arregloEnteros[longitud(arregloEnteros)] = 6;
```

Eliminación:

```
// Eliminar el tercer elemento del arreglo
para i desde 2 hasta longitud(arregloEnteros) - 1 con paso 1
    arregloEnteros[i] = arregloEnteros[i + 1];
fin para
```

Ordenamiento:

```
// Ordenar el arreglo de manera ascendente
para i desde 0 hasta longitud(arregloEnteros) - 2 con paso 1
    para j desde 0 hasta longitud(arregloEnteros) - i - 2 con paso 1
        si arregloEnteros[j] > arregloEnteros[j + 1] entonces
            // Intercambiar elementos si están en el orden incorrecto
            intercambiar(arregloEnteros[j], arregloEnteros[j + 1]);
        fin si
    fin para
fin para
```

```
// Ordenamiento descendente
para i desde 0 hasta longitud(numeros) - 2 con paso 1
    para j desde 0 hasta longitud(numeros) - i - 2 con paso 1
        si numeros[j] < numeros[j + 1] entonces
            // Intercambiar elementos si están en el orden incorrecto
            intercambiar(numeros[j], numeros[j + 1]);
        fin si
    fin para
fin para
```